3. 윈도우에서 파이썬 사용하기¶

이 문서는 Microsoft 윈도우에서 파이썬을 사용할 때 알아야 할 윈도우 특정 동작에 대한 개요를 제공하는 것을 목표로 합니다.

대부분의 유닉스 시스템의 서비스와 달리, 윈도우에는 시스템 지원 파이썬 설치가 포함되어 있지 않습니다. 파이썬을 사용할 수 있도록, CPython 팀은 수년 동안 모든 배포판에 대해 윈도우 설치 프로그램(MSI 패키지)을 컴파일했습니다. 이러한 설치 프로그램은 주로 단일 사용자가 사용하는 핵심 인터프리터와 라이브러리와 함께 사용자별 파이썬 설치를 추가하기 위한 것입니다. 설치 프로그램은 또한 단일 시스템의 모든 사용자를 위해 설치할 수 있으며, 응용 프로그램 로컬 배포를 위해 별도의 ZIP 파일이 제공됩니다.

PEP 11에 지정된 대로, 파이썬 릴리스는 Microsoft가 플랫폼이 확장된 지원을 받는 것으로 간주하는 윈도우 플랫폼만 지원합니다. 이것은 파이썬 3.9가(이) 윈도우 8.1 이상을 지원함을 뜻합니다. 윈도우 7 지원이 필요하면 파이썬 3.8을 설치하십시오.

윈도우에서 사용할 수 있는 여러 가지 설치 프로그램이 있으며, 각각 나름의 장단점이 있습니다.

전체 설치 프로그램은 모든 구성 요소를 포함하며 모든 종류의 프로젝트에 파이썬을 사용하는 개발자에게 가장 적합한 옵션입니다.

Microsoft Store 패키지는 스크립트와 패키지를 실행하고 IDLE이나 다른 개발 환경을 사용하는 데 적합한 파이썬의 간단한 설치입니다. 윈도우 10이 필요하지만, 다른 프로그램을 손상하지 않고 안전하게 설치할 수 있습니다. 또한 파이썬과 그 도구를 시작하기 위한 많은 편리한 명령을 제공합니다.

nuget.org 패키지는 지속적 통합 시스템(continuous integration systems)을 위한 경량 설치입니다. 파이썬 패키지를 빌드하거나 스크립트를 실행하는 데 사용할 수 있지만, 업데이트할 수 없으며 사용자 인터페이스 도구가 없습니다.

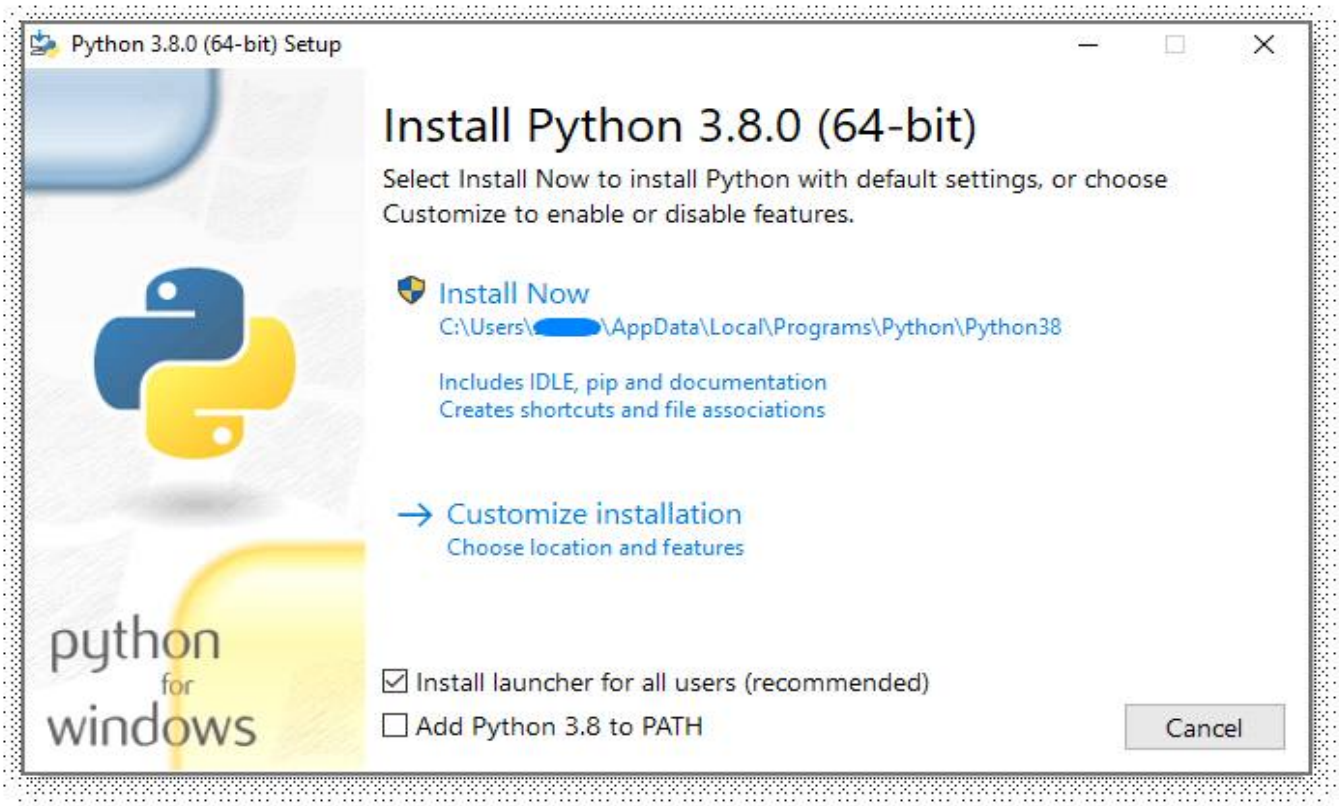
내장 가능한 패키지는 큰 응용 프로그램에 내장하기에 적합한 파이썬의 최소 패키지입니다.

3.1. 전체 설치 프로그램

3.1.1. 설치 단계

네 개의 파이썬 3.9 설치 프로그램을 다운로드할 수 있습니다 - 각각 32비트와 64비트 버전의 인터프리터를 지원하는 두 가지. *웹 설치 프로그램(web installer)*은 작은 초기 다운로드이며, 필요한 구성 요소를 자동으로 다운로드합니다. *오프라인 설치 프로그램(offline installer)*에는 기본 설치에 필요한 구성 요소가 포함되어 있으며 선택적 기능을 위해서만 인터넷 연결이 필요합니다. 설치 중 다운로드를 피하는 다른 방법은 다운로드 없이 설치하기를 참조하십시오.

설치 프로그램을 시작한 후, 두 가지 옵션 중 하나를 선택할 수 있습니다:



《Install Now》를 선택하면:
관리자(administrator)일 필요는 *없습니다*(C 런타임 라이브러리에 대한 시스템 업데이트가 필요하거나 모든 사용자를 위해 윈도우 용 파이썬 런처를 설치하지 않는 한)
파이썬이 사용자 디렉터리에 설치됩니다
윈도우 용 파이썬 런처는 첫 페이지 하단의 옵션에 따라 설치됩니다
표준 라이브러리, 테스트 스위트, 런처 및 pip가 설치됩니다
선택하면, 설치 디렉터리가 PATH에 추가됩니다
바로 가기는 현재 사용자에게만 표시됩니다.

《Customize installation》을 선택하면 설치할 기능, **설치 위치** 및 다른 옵션이나 설치 후 작업을 선택할 수 있습니다. 디버깅 심볼이나 바이너리를 설치하려면, 이 옵션을 사용해야 합니다.

모든 사용자 설치를 수행하려면, 《Customize installation》을 선택해야 합니다. 이 경우:
관리자 자격 증명이나 승인을 제공해야 할 수 있습니다.

파이썬은 Program Files 디렉터리에 설치됩니다
윈도우 용 파이썬 런처는 Windows 디렉터리에 설치됩니다
설치 중에 선택적 기능을 선택할 수 있습니다
표준 라이브러리는 바이트 코드로 사전 컴파일될 수 있습니다

선택하면, 설치 디렉터리가 시스템 PATH에 추가됩니다
모든 사용자가 바로 가기를 사용할 수 있습니다

3.1.2. MAX_PATH 제한 제거하기

윈도우는 역사적으로 경로 길이를 260자로 제한했습니다. 이는 이보다 긴 경로는 결정(resolve)되지 않고 에러가 발생함을 의미합니다.
최신 버전의 윈도우에서는, 이 제한을 약 32,000자로 확장할 수 있습니다.
관리자는 《Enable Win32 long paths》 그룹 정책을 활성화하거나, 레지스트리 키 HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem에서 LongPathsEnabled를 1로 설정해야 합니다.
이는 open() 함수, os 모듈 및 대부분의 다른 경로 기능이 260자보다 긴 경로를 받아들이고 반환할 수 있도록 합니다.
위 옵션을 변경한 후에는, 추가 구성이 필요하지 않습니다.
버전 3.6에서 변경: 긴 경로에 대한 지원이 파이썬에서 활성화되었습니다.

3.1.3. UI 없이 설치하기

설치 프로그램 UI에서 사용할 수 있는 모든 옵션은 명령 줄에서도 지정할 수 있어서, 스크립팅 된 설치 프로그램이 사용자 상호 작용 없이 여러 컴퓨터에서 설치를 복제할 수 있도록 합니다. 이러한 옵션은 일부 기본값을 변경하기 위해 UI를 억제하지 않고 설정할 수도 있습니다.
설치 프로그램 UI를 완전히 숨기고 파이썬을 자동으로 설치하려면, /quiet 옵션을 전달하십시오. 사용자 상호 작용을 건너뛰지만, 진행률과 에러를 계속 표시하려면, /passive 옵션을 전달하십시오. /uninstall 옵션을 전달하면 파이썬 제거를 즉시 시작합니다 - 프롬프트가 표시되지 않습니다.

다른 모든 옵션은 name=value로 전달됩니다. 여기서 value는 일반적으로 기능을 비활성화하려면 0, 기능을 활성화하려면 1 또는 경로입니다. 사용 가능한 옵션의 전체 목록은 다음과 같습니다.

이름	설명	기본값
InstallAllUsers	시스템 전체 설치를 수행합니다.	0
TargetDir	설치 디렉터리	InstallAllUsers 에 따라 선택됩니다
DefaultAllUsersTargetDir	모든 사용자 설치를 위한 기본 설치 디렉터리	%ProgramFiles%\Python X.Y 또는 %ProgramFiles(x86)%\Python X.Y
DefaultJustForMeTargetDir	현재 사용자 전용 설치를 위한 기본 설치 디렉터리	%LocalAppData%\Programs\PythonXY 또는 %LocalAppData%\Programs\PythonX

		Y-32 또는 %LocalAppData%\Programs\PythonXY-64
DefaultCustomTargetDir	UI에 표시되는 기본 사용자 지정 설치 디렉터리	(비어있음)
AssociateFiles	런처도 설치되었으면 파일 연결을 만듭니다.	1
CompileAll	모든 .py 파일을 .pyc로 컴파일합니다.	0
PrependPath	설치 및 Scripts 디렉터리를 PATH에 추가하고 .PY를 PATHEXT에 추가합니다.	0
Shortcuts	설치되면, 인터프리터, 설명서 및 IDLE에 대한 바로 가기를 만듭니다.	1
Include_doc	파이썬 매뉴얼을 설치합니다	1
Include_debug	디버그 바이너리를 설치합니다	0
Include_dev	개발자 헤더와 라이브러리를 설치합니다	1
Include_exe	python.exe와 관련 파일들을 설치합니다	1
Include_launcher	윈도우 용 파이썬 런처를 설치합니다.	1
InstallLauncherAllUsers	모든 사용자를 위해 윈도우 용 파이썬 런처를 설치합니다.	1
Include_lib	표준 라이브러리와 확장 모듈을 설치합니다	1
Include_pip	번들로 제공되는 pip와 setuptools를 설치합니다	1
Include_symbols	디버깅 심볼(*.pdb)을 설치합니다	0
Include_tcltk	Tcl/Tk 지원과 IDLE을 설치합니다	1
Include_test	표준 라이브러리 테스트 스위트를 설치합니다	1
Include_tools	유틸리티 스크립트를 설치합니다	1
LauncherOnly	런처만 설치합니다. 이것은 대부분의 다른 옵션보다 우선합니다.	0
SimpleInstall	대부분의 설치 UI를 비활성화합니다	0
SimpleInstallDescription	단순화된 설치 UI를 사용할 때 표시할 사용자 정의 메시지.	(비어있음)

예를 들어 기본, 시스템 전체 파이썬 설치를 조용히 설치하려면, (관리자 권한 명령 프롬프트에서) 다음 명령을 사용할 수 있습니다:

python-3.9.0.exe /quiet InstallAllUsers=1 PrependPath=1 Include_test=0

사용자가 테스트 스위트 없이 파이썬의 개인용 사본을 쉽게 설치할 수 있도록, 다음 명령으로 바로 가기를 제공할 수 있습니다. 이렇게 하면 단순화된 초기 페이지가 표시되고 사용자 정의가 허용되지 않습니다:

python-3.9.0.exe InstallAllUsers=0 Include_launcher=0 Include_test=0
SimpleInstall=1 SimpleInstallDescription="Just for me, no test suite."

(런처를 생략하면 파일 연결도 생략되며, 런처가 포함된 시스템 전체 설치가 있을 때, 사용자

별 설치에만 권장됨에 유의하십시오.)
위에 나열된 옵션은 실행 파일과 함께 unattend.xml이라는 파일로 제공될 수도 있습니다. 이 파일은 옵션과 값 목록을 지정합니다. 값이 어트리뷰트로 제공되면, 가능한 경우 숫자로 변환됩니다. 엘리먼트 텍스트로 제공된 값은 항상 문자열로 남아 있습니다. 이 예제 파일은 이전 예제와 같은 옵션을 설정합니다:

```
<Options>
    <Option Name="InstallAllUsers" Value="no" />
    <Option Name="Include_launcher" Value="0" />
    <Option Name="Include_test" Value="no" />
    <Option Name="SimpleInstall" Value="yes" />
    <Option Name="SimpleInstallDescription">Just for me, no test suite</Option>
</Options>
```

3.1.4. 다운로드 없이 설치하기

파이썬의 일부 기능은 초기 설치 프로그램 다운로드에 포함되어 있지 않아서, 이러한 기능을 선택하면 인터넷 연결이 필요할 수 있습니다. 이러한 요구를 피하고자, 필요에 따라 모든 가능한 구성 요소를 내려받아 선택한 기능과 관계없이 더는 인터넷 연결이 필요하지 않은 완전한 *레이아웃*을 만들 수 있습니다. 이 다운로드는 필요한 것보다 클 수 있지만, 많은 수의 설치를 수행할 경우 로컬로 캐시 된 사본을 보유하는 것이 매우 유용합니다.
가능한 모든 파일을 다운로드하려면 명령 프롬프트에서 다음 명령을 실행하십시오. python-3.9.0.exe를 설치 프로그램의 실제 이름으로 대체하고, 같은 이름을 가진 파일 간의 충돌을 방지하기 위해 자체 디렉터리에 레이아웃을 만들어야 합니다.

```
python-3.9.0.exe /layout [optional target directory]
```

/quiet 옵션을 지정하여 진행률 표시를 숨길 수도 있습니다.

3.1.5. 설치 수정하기

일단 파이썬이 설치되면, 윈도우의 일부인 Programs and Features 도구를 통해 기능을 추가하거나 제거할 수 있습니다. Python 항목을 선택하고 《Uninstall/Change》를 선택하여 유지 관리 모드로 설치 프로그램을 엽니다.
《Modify》는 체크 박스를 수정하여 기능을 추가하거나 제거하도록 합니다 - 변경되지 않은 체크 박스는 아무것도 설치하거나 제거하지 않습니다. 이 모드에서는 설치 디렉터리와 같은 일부 옵션을 변경할 수 없습니다; 이를 수정하려면 파이썬을 완전히 제거한 다음 다시 설치해야 합니다.
《Repair》는 현재 설정을 사용하여 설치되어야 하는 모든 파일을 확인하고 제거되거나 수정된 파일을 대체합니다.
《Uninstall》은 파이썬을 완전히 제거하는데, Programs and Features에 자체 항목이 있는 윈도우 용 파이썬 런처는 제외합니다.

3.2. Microsoft Store 패키지

버전 3.7.2에 추가.

Microsoft Store 패키지는 쉽게 설치할 수 있는 파이썬 인터프리터로, 주로 예를 들어 학생의 대화형 사용을 목적으로 합니다.

패키지를 설치하려면, 최신 윈도우 10 업데이트가 있는지 확인하고 《Python 3.9》를(을) 위한 Microsoft Store 앱을 검색하십시오. 선택한 앱이 Python Software Foundation에서 게시되었는지 확인하고, 설치합니다.

경고

파이썬은 Microsoft Store에서 항상 무료로 제공됩니다. 비용을 지불하라는 요청을 받으면, 올바른 패키지를 선택하지 않은 것입니다.

설치 후, 파이썬은 Start에서 찾아서 시작할 수 있습니다. 또는, 모든 명령 프롬프트 또는 PowerShell 세션에서 python을 입력하여 사용할 수 있습니다. 또한, pip나 idle을 입력하여 pip와 IDLE을 사용할 수 있습니다. IDLE은 Start에서도 찾을 수 있습니다.

세 가지 명령 모두 버전 번호 접미사와 함께 사용할 수 있습니다, 예를 들어, python.exe뿐만 아니라 python3.exe와 python3.x.exe (여기서 3.x는 여러분이 시작하려는 특정 버전입니다, 가령 3.9). Start를 통해 《Manage App Execution Aliases》를 열어 각 명령과 연결된 파이썬 버전을 선택합니다. pip와 idle이 선택된 python 버전과 일치하도록 하는 것이 좋습니다.

python -m venv로 가상 환경을 만들고 활성화하여 정상적으로 사용할 수 있습니다.

다른 버전의 파이썬을 설치하고 PATH 변수에 추가했으면, Microsoft Store에서 제공하는 것이 아니라 그것이 python.exe로 사용될 수 있습니다. 새 설치에 액세스하려면, python3.exe나 python3.x.exe를 사용하십시오.

py.exe 가 이 파이썬 설치를 감지하지만, 기존 설치프로그램으로부터의 설치를 선호합니다.

파이썬을 제거하려면, Settings를 열고 Apps and Features를 사용하거나, Start에서 Python을 찾은 다음 마우스 오른쪽 단추를 클릭하여 Uninstall 을 선택합니다. 제거하면 이 파이썬 설치에 직접 설치한 모든 패키지가 제거되지만, 가상 환경은 제거되지 않습니다.

3.2.1. 알려진 문제

Microsoft Store 앱에 대한 제한으로 인해, 파이썬 스크립트에는 TEMP와 레지스트리와 같은 공유 위치에 대한 전체 쓰기 액세스 권한이 없을 수 있습니다. 대신, 개인 사본에 기록됩니다. 스크립트가 공유 위치를 수정해야 하면, 전체 설치 프로그램을 설치해야 합니다.

이러한 제한에 대한 기술적 기반에 대한 자세한 내용은, 현재 docs.microsoft.com/en-us/windows/msix/desktop/desktop-to-uwp-behind-the-scenes에 있는 패키지화 된 완전 신뢰 앱에 대한 Microsoft의 설명서를 참조하십시오.

3.3. nuget.org 패키지

버전 3.5.2에 추가.

nuget.org 패키지는 시스템 전체에 파이썬이 설치되지 않은 지속적인 통합과 빌드 시스템에 사용하기 위한 축소된 크기의 파이썬 환경입니다. 너겟은 《.NET 용 패키지 관리자》이지만, 빌드 타임 도구가 포함된 패키지에서도 완벽하게 작동합니다.

너겟 사용에 대한 최신 정보를 보려면 nuget.org를 방문하십시오. 다음은 파이썬 개발자에게 충분한 요약입니다.

nuget.exe 명령 줄 도구는, 예를 들어 curl이나 PowerShell을 사용하여 https://aka.ms/nugetclidl에서 직접 다운로드할 수 있습니다. 이 도구를 사용하면 다음과 같이 64비트나 32비트 컴퓨터용 파이썬의 최신 버전이 설치됩니다:

```
python-3.9.0.exe /layout [optional target directory]
nuget.exe install python -ExcludeVersion -OutputDirectory .
nuget.exe install pythonx86 -ExcludeVersion -OutputDirectory .
```

특정 버전을 선택하려면, -Version 3.x.y를 추가하십시오. 출력 디렉터리는 .에서 변경될 수 있으며, 패키지는 하위 디렉터리에 설치됩니다. 기본적으로, 하위 디렉터리의 이름은 패키지와 같으며, -ExcludeVersion 옵션이 없으면 이 이름에 설치된 특정 버전이 포함됩니다. 하위 디렉터리에는 파이썬 설치가 포함된 tools 디렉터리가 있습니다:

```
# Without -ExcludeVersion
> .\python.3.5.2\tools\python.exe -V
Python 3.5.2

# With -ExcludeVersion
> .\python\tools\python.exe -V
Python 3.5.2
```

일반적으로, 너겟 패키지는 업그레이드할 수 없으며, 최신 버전을 나란히 설치하고 전체 경로를 사용하여 참조해야 합니다. 또는, 패키지 디렉터리를 수동으로 삭제하고 다시 설치하십시오. 많은 CI 시스템은 빌드 간에 파일을 보존하지 않으면 이 작업을 자동으로 수행합니다.

tools 디렉터리와 함께 build\native 디렉터리가 있습니다. 여기에는 파이썬 설치를 참조하기 위해 C++ 프로젝트에서 사용할 수 있는 MSBuild 속성 파일 python.props가 포함되어 있습니다. 설정을 포함하면 빌드에서 자동으로 헤더와 임포트 라이브러리를 사용합니다.

nuget.org의 패키지 정보 페이지는 64비트 버전의 경우 www.nuget.org/packages/python 이고 32비트 버전의 경우 www.nuget.org/packages/pythonx86입니다.

3.4. 내장 가능한 패키지

버전 3.5에 추가.

내장된 배포는 최소 파이썬 환경을 포함하는 ZIP 파일입니다. 최종 사용자가 직접 액세스하기 보다는, 다른 응용 프로그램의 일부로 작동하기 위한 것입니다.
추출되면, 내장된 배포는 환경 변수, 시스템 레지스트리 설정 및 설치된 패키지를 포함하여 사용자 시스템에서 (거의) 완전히 격리됩니다. 표준 라이브러리는 ZIP에 미리 컴파일되고 최적화된 .pyc 파일로 포함되며, python3.dll, python37.dll, python.exe 및 pythonw.exe가 모두 제공됩니다. Tcl/tk (Idle과 같은 모든 종속 항목을 포함하여), pip 및 파이썬 설명서는 포함되지 않습니다.

참고

내장된 배포에는 Microsoft C Runtime이 포함되어 있지 않으며 이를 제공하는 것은 응용 프로그램 설치 프로그램의 책임입니다. 런타임은 이전에 사용자 시스템에 이미 설치되었거나 윈도우 업데이트를 통해 자동으로 설치되었을 수 있으며, 시스템 디렉터리에서 ucrtbase.dll을 찾아서 감지할 수 있습니다.

제삼자 패키지는 내장된 배포와 함께 응용 프로그램 설치 프로그램이 설치해야 합니다. 일반 파이썬 설치처럼 종속성을 관리하기 위해 pip를 사용하는 것은 이 배포에서 지원되지 않지만, 주의를 기울이면 자동 업데이트를 위해 pip를 포함하고 사용할 수 있습니다. 일반적으로, 제삼자 패키지는 개발자가 사용자에게 업데이트를 제공하기 전에 최신 버전과의 호환성을 보장할 수 있도록 응용 프로그램의 일부로 처리되어야 합니다 (《벤더링(vendoring)》).
이 배포에 권장되는 두 가지 사용 사례가 아래에 설명되어 있습니다.

3.4.1. 파이썬 응용 프로그램

파이썬으로 작성된 응용 프로그램이 반드시 사용자가 그 사실을 인식하도록 할 필요는 없습니다. 이 경우 내장된 배포를 사용하여 설치 패키지에 파이썬의 내부 버전을 포함할 수 있습니다. 얼마나 투명해야 하는지(또는 반대로, 얼마나 전문적으로 보여야 하는지)에 따라, 두 가지 옵션이 있습니다.
특수 실행 파일을 런처로 사용하려면 약간의 코딩이 필요하지만, 사용자에게 가장 투명한 경험을 제공합니다. 사용자 정의된 런처를 사용하면, 프로그램이 파이썬에서 실행되고 있다는 명백한 표시가 없습니다: 아이콘을 사용자 정의하고, 회사와 버전 정보를 지정할 수 있으며 파일 연결이 제대로 작동합니다. 대부분의 경우, 사용자 정의 런처는 하드 코딩된 명령 줄을 사용하여 Py_Main을 호출할 수 있어야 합니다.
더 간단한 방법은 필요한 명령 줄 인자를 사용하여 python.exe나 pythonw.exe를 직접 호출하는 배치 파일이나 생성된 바로 가기를 제공하는 것입니다. 이 경우, 응용 프로그램은 실제 이름이 아닌 파이썬으로 표시되며, 사용자는 실행 중인 다른 파이썬 프로세스나 파일 연결과 구별하는 데 어려움을 겪을 수 있습니다.
후자의 접근 방식에서는, 패키지를 파이썬 실행 파일과 함께 디렉터리로 설치하여 경로에서 사용할 수 있도록 해야 합니다. 특수 런처를 사용하면, 응용 프로그램을 시작하기 전에 검색 경로를 지정할 수 있어서 패키지를 다른 위치에 배치할 수 있습니다.

3.4.2. 파이썬 내장하기

네이티브 코드로 작성된 응용 프로그램에는 종종 어떤 형태의 스크립팅 언어가 필요하며, 내장된 파이썬 배포를 이러한 목적으로 사용할 수 있습니다. 일반적으로, 대부분의 응용 프로그램은 네이티브 코드로 되어 있으며, 일부가 python.exe를 호출하거나 python3.dll을 직접 사용합니다. 두 경우 모두, 내장된 배포를 응용 프로그램 설치의 하위 디렉터리로 추출하면 로드할 수 있는 파이썬 인터프리터를 제공하기에 충분합니다.
응용 프로그램 사용과 마찬가지로, 인터프리터를 초기화하기 전에 검색 경로를 지정할 기회가 있어서 패키지를 임의의 위치에 설치할 수 있습니다. 그 외에는, 내장된 배포와 일반 설치를 사용하는 것 간에 근본적인 차이점은 없습니다.

3.5. 대체 번들

표준 CPython 배포 외에도, 추가 기능을 포함하는 수정된 패키지가 있습니다. 다음은 많이 사용되는 버전과 주요 기능 목록입니다:

ActivePython

다중 플랫폼 호환성, 설명서, PyWin32가 있는 설치 프로그램

Anaconda

인기 있는 과학 모듈(가령 numpy, scipy 및 pandas)과 conda 패키지 관리자.

Canopy

편집기와 기타 개발 도구가 있는 《포괄적인 파이썬 분석 환경》.

WinPython

사전 빌드된 과학 패키지와 패키지 빌드를 위한 도구가 포함된 윈도우 전용 배포.

이러한 패키지들은 최신 버전의 파이썬이나 기타 라이브러리를 포함하지 않을 수 있으며, 핵심 파이썬 팀에서 유지 관리하거나 지원하지 않음에 유의하십시오.

3.6. 파이썬 구성하기

명령 프롬프트에서 파이썬을 편리하게 실행하려면, 윈도우에서 일부 기본 환경 변수를 변경하는 것을 고려할 수 있습니다. 설치 프로그램이 PATH와 PATHEXT 변수를 구성하는 옵션을 제공하지만, 이는 시스템 전체의 단일 설치에서만 신뢰할 수 있습니다. 여러 버전의 파이썬을 정기적으로 사용하면, 윈도우 용 파이썬 런처 사용을 고려하십시오.

3.6.1. 보충 설명: 환경 변수 설정하기

윈도우에서는 환경 변수를 사용자 수준과 시스템 수준 모두에서 영구적으로 구성하거나, 명령 프롬프트에서 일시적으로 구성할 수 있습니다.
환경 변수를 임시로 설정하려면, 명령 프롬프트를 열고 **set** 명령을 사용하십시오:

```
C:\>set PATH=C:\Program Files\Python 3.9;%PATH%
C:\>set PYTHONPATH=%PYTHONPATH%;C:\My_python_lib
C:\>python
```

이러한 변경은 해당 콘솔에서 실행되는 모든 추가 명령에 적용되며, 콘솔에서 시작된 모든 응용 프로그램에 상속됩니다.
백분율 기호 안에 변수 이름을 포함하면 기존 값으로 확장되어, 시작이나 끝부분에 새 값을 추가할 수 있습니다. **python.exe**를 포함하는 디렉터리를 시작 부분에 추가하여 PATH를 수정하는 것은 올바른 버전의 파이썬이 실행되도록 하는 일반적인 방법입니다.

기본 환경 변수를 영구적으로 수정하려면, Start를 클릭하고 〈edit environment variables'를 검색하거나, 시스템 속성, Advanced system settings를 열고 Environment Variables 버튼을 클릭합니다. 이 대화 상자에서, 사용자와 시스템 변수를 추가하거나 수정할 수 있습니다. 시스템 변수를 변경하려면 컴퓨터에 제한 없이 액세스해야 합니다 (즉 관리자 권한).

참고

윈도우는 사용자 변수를 시스템 변수 *뒤에* 이어붙이므로, PATH를 수정할 때 예기치 않은 결과가 발생할 수 있습니다.

PYTHONPATH 변수는 모든 버전의 파이썬 2와 파이썬 3에서 사용되므로, 설치된 모든 파이썬 버전과 호환되는 코드만 포함하지 않는 한 이 변수를 영구적으로 구성해서는 안 됩니다.

더 보기

https://www.microsoft.com/en-us/wdsi/help/folder-variables

윈도우 NT의 환경 변수

https://technet.microsoft.com/en-us/library/cc754250.aspx

환경 변수를 임시로 수정하기 위한, SET 명령

https://technet.microsoft.com/en-us/library/cc755104.aspx

환경 변수를 영구적으로 수정하기 위한, SETX 명령

https://support.microsoft.com/en-us/help/310519/how-to-manage-environment-variables-in-windows-xp

윈도우 XP에서 환경 변수를 관리하는 방법

https://www.chem.gla.ac.uk/~louis/software/faq/q1.html

환경 변수 설정하기, Louis J. Farrugia

3.6.2. 파이썬 실행 파일 찾기

버전 3.5에서 변경.

자동으로 만들어진 파이썬 인터프리터를 위한 시작 메뉴 항목을 사용하는 것 외에도, 명령 프롬프트에서 파이썬을 시작하고 싶을 수 있습니다. 설치 프로그램에는 이를 설정할 수 있는 옵션이 있습니다.

설치 프로그램의 첫 페이지에서, 《Add Python to PATH》라는 옵션을 선택하여 설치 프로그램이 PATH에 설치 위치를 추가하도록 할 수 있습니다. Scripts\ 폴더의 위치도 추가됩니다.

이는 인터프리터를 실행하기 위해 **python**을 입력하고, 패키지 설치 프로그램을 실행하려면 **pip**를 입력할 수 있도록 합니다. 따라서, 명령 줄 옵션으로 스크립트를 실행할 수도 있습니다, 명령 줄 설명서를 참조하십시오.

설치 시 이 옵션을 활성화하지 않았으면, 언제든지 설치 프로그램을 다시 실행하고, Modify를 선택한 다음, 활성화할 수 있습니다. 또는, 보충 설명: 환경 변수 설정하기의 지침을 사용하여 PATH를 수동으로 수정할 수 있습니다. 파이썬 설치 디렉터리를 포함하도록 PATH 환경 변수를 설정해야 하며, 다른 항목과 세미콜론으로 구분됩니다. 예제 변수는 다음과 같습니다 (처음 두 항목이 이미 존재한다고 가정합니다):
C:\WINDOWS\system32;C:\WINDOWS;C:\Program Files\Python 3.9

3.7. UTF-8 모드

버전 3.7에 추가.
윈도우는 여전히 시스템 인코딩에 레거시 인코딩을 사용합니다 (ANSI 코드 페이지). 파이썬은 이를 텍스트 파일의 기본 인코딩에 이를 사용합니다 (예를 들어 locale.getpreferredencoding()).
UTF-8은 인터넷과 WSL(Windows Subsystem for Linux)을 포함한 대부분의 유닉스 시스템에서 널리 사용되기 때문에 문제가 발생할 수 있습니다.
UTF-8 모드를 사용하여 기본 텍스트 인코딩을 UTF-8로 변경할 수 있습니다. -X utf8 명령 줄 옵션이나 PYTHONUTF8=1 환경 변수를 통해 UTF-8 모드를 활성화할 수 있습니다.
UTF-8 모드를 활성화하려면 PYTHONUTF8을, 환경 변수를 수정하는 방법은 보충 설명: 환경 변수 설정하기를 참조하십시오.
UTF-8 모드가 활성화될 때:
locale.getpreferredencoding() 은 시스템 인코딩 대신 'UTF-8'을 반환합니다. 이 함수는 open(), Popen, Path.read_text() 등 여러 곳에서 기본 텍스트 인코딩으로 사용됩니다.
sys.stdin, sys.stdout 및 sys.stderr는 모두 UTF-8을 텍스트 인코딩으로 사용합니다.
《mbcs》 코덱을 통해 시스템 인코딩을 계속 사용할 수 있습니다.
기본 환경 변수에 PYTHONUTF8=1을 추가하면 시스템의 모든 파이썬 3.7+ 응용 프로그램에 영향을 줍니다. 레거시 시스템 인코딩에 의존하는 파이썬 3.7+ 응용 프로그램이 있으면 환경 변수를 임시로 설정하거나 -X utf8 명령 줄 옵션을 사용하는 것이 좋습니다.
참고

UTF-8 모드가 비활성화된 경우에도, 파이썬은 윈도우에서 기본적으로 다음을 위해 UTF-8을 사용합니다:

표준 I/O를 포함한 콘솔 I/O (자세한 내용은 PEP 528을 참조하십시오).

파일 시스템 인코딩 (자세한 내용은 PEP 529를 참조하십시오).

3.8. 윈도우 용 파이썬 런처

버전 3.3에 추가.

윈도우 용 파이썬 런처는 다양한 파이썬 버전을 찾고 실행하는 데 도움이 되는 유틸리티입니다. 스크립트(또는 명령 줄)가 특정 파이썬 버전에 대한 선호를 나타내도록 허용하고, 해당 버전을 찾아 실행합니다.

PATH 변수와 달리, 런처는 가장 적합한 파이썬 버전을 올바르게 선택합니다. 시스템 전체 설치보다 사용자별 설치를 선호하며, 가장 최근에 설치된 버전을 사용하기보다 언어 버전별로 순서를 매깁니다.

런처는 원래 PEP 397에서 지정되었습니다.

3.8.1. 시작하기

3.8.1.1. 명령 줄에서

버전 3.6에서 변경.

시스템 전체에 파이썬 3.3 이상을 설치하면 PATH에 런처를 넣습니다. 런처는 사용 가능한 모든 파이썬 버전과 호환되므로, 어떤 버전이 설치되어 있는지는 중요하지 않습니다. 런처를 사용할 수 있는지 확인하려면, 명령 프롬프트에서 다음 명령을 실행하십시오:

```
py
```

설치한 최신 버전의 파이썬이 시작되어야 합니다 - 정상적으로 종료할 수 있으며, 지정된 추가 명령 줄 인자가 파이썬으로 직접 전송됩니다.

여러 버전의 파이썬이 설치되어있으면 (예를 들어, 2.7과 3.9) 파이썬 3.9 시작되는 것을 알 수 있을 겁니다 - 파이썬 2.7을 시작하려면, 다음 명령을 시도하십시오:

```
py -2.7
```

설치한 파이썬 2.x의 최신 버전을 원하면, 다음 명령을 시도하십시오:

```
py -2
```

최신 버전의 파이썬 2.x가 시작되어야 합니다.

다음 에러가 표시되면, 런처가 설치되지 않은 것입니다:

```
C:\>setPYTHONPATH=%PYTHONPATH%;C:\My_python_lib
C:\>python
C:\WINDOWS\system32;C:\WINDOWS;C:\Program Files\Python 3.9
'py' is not recognized as an internal or external command,
operable program or batch file.
```

사용자별 파이썬 설치는 설치 시 옵션을 선택하지 않는 한 PATH에 런처를 추가하지 않습니다.

3.8.1.2. 가상 환경

버전 3.5에 추가.

런처가 명시적인 파이썬 버전 지정 없이 실행되고, 가상 환경(표준 라이브러리 venv 모듈이나 외부 virtualenv 도구로 생성된)이 활성화되었으면, 런처는 전역 인터프리터가 아닌 가상 환경의 인터프리터를 실행합니다. 전역 인터프리터를 실행하려면 가상 환경을 비활성화하거나, 전역 파이썬 버전을 명시적으로 지정하십시오.

3.8.1.3. 스크립트에서

테스트 파이썬 스크립트를 만들어 봅시다 - 다음 내용으로 hello.py라는 파일을 만듭니다.

```
#! python
import sys
sys.stdout.write("hello from Python %s\n" % (sys.version,))
```

최신 파이썬 2.x 설치의 버전 번호가 인쇄됨을 알 수 있습니다.

```
py hello.py
```

이제 첫 번째 줄을 다음과 같이 변경합니다:

```
#! python3
```

명령을 다시 실행하면 이제 최신 파이썬 3.x 정보가 인쇄됩니다. 위의 명령 줄 예제와 마찬가지로, 더 명시적인 버전 한정자를 지정할 수 있습니다. 파이썬 2.6이 설치되어 있다고 가정하고, 첫 번째 줄을 #! python2.6으로 변경하면 2.6 버전 정보가 인쇄됩니다.

대화 형 사용과 달리, 장식 없는 《python》은 설치된 파이썬 2.x의 최신 버전을 사용합니다. 이는 이전 버전과의 호환성과 유닉스와의 호환성을 위한 것입니다, 여기서 python 명령은 일반적으로 파이썬 2를 참조합니다.

3.8.1.4. 파일 연결에서

런처는 설치 시 파이썬 파일(즉 .py, .pyw, .pyc 파일)과 연결되어 있어야 합니다. 즉, 윈도우 탐색기에서 이러한 파일 중 하나를 더블 클릭하면 런처가 사용되므로, 위에서 설명한 것과 같은 기능을 사용하여 스크립트에서 사용해야 하는 버전을 지정할 수 있습니다.

이것의 주요 이점은 첫 번째 줄의 내용에 따라 단일 런처가 동시에 여러 파이썬 버전을 지원할 수 있다는 것입니다.

3.8.2. 셔뱅 줄

스크립트 파일의 첫 번째 줄이 **#!로 시작하면 《셔뱅(shebang)》줄이라고 합니다.** 리눅스와 기타 유닉스 류 운영 체제는 이러한 줄을 기본적으로 지원하며 스크립트 실행 방법을 나타내기

위해 이러한 시스템에서 일반적으로 사용됩니다. 이 런처를 사용하면 윈도우에서 파이썬 스크립트로 같은 기능을 사용할 수 있으며 위의 예제는 그 사용법을 보여줍니다.
파이썬 스크립트의 셔뱅 줄을 유닉스와 윈도우 간에 이식성 있도록 하기 위해, 이 런처는 사용할 인터프리터를 지정하는 여러 〈가상〉 명령을 지원합니다. 지원되는 가상 명령은 다음과 같습니다:

#!/usr/bin/env python
#!/usr/bin/python
#!/usr/local/bin/python
#!python

예를 들어, 스크립트의 첫 번째 줄이 다음과 같이 시작하면

#! /usr/bin/python

기본 파이썬을 찾아서 사용합니다. 유닉스에서 작동하도록 작성된 많은 파이썬 스크립트에는 이미 이 줄이 있어서, 수정하지 않고도 이러한 스크립트를 런처에서 사용할 수 있습니다. 윈도우에서 유닉스에서 유용할 새 스크립트를 작성하면, /usr로 시작하는 셔뱅 줄 중 하나를 사용해야 합니다.
위의 모든 가상 명령은 명시적 버전(주 버전 만 또는 주 버전과 부 버전)으로 접미사를 붙일 수 있습니다. 또한 부 버전 뒤에 《-32》를 추가하여 32비트 버전을 요청할 수 있습니다. 즉 /usr/bin/python2.7-32는 32비트 파이썬 2.7의 사용을 요청합니다.

버전 3.7에 추가: 파이썬 런처 3.7부터는 《-64》 접미사로 64비트 버전을 요청할 수 있습니다. 또한 부 버전 없이 주 버전과 아키텍처를 지정할 수 있습니다 (즉 /usr/bin/python3-64).
/usr/bin/env 형식의 셔뱅 줄에는 또 다른 특별한 성질이 있습니다. 설치된 파이썬 인터프리터를 찾기 전에, 이 형식은 파이썬 실행 파일을 PATH에서 검색합니다. 이것은 PATH 검색을 수행하는 유닉스 env 프로그램의 동작에 해당합니다.

3.8.3. 셔뱅 줄의 인자

셔뱅 줄은 또한 파이썬 인터프리터에 전달할 추가 옵션을 지정할 수 있습니다. 예를 들어, 다음과 같은 셔뱅 줄이 있으면:

#! /usr/bin/python -v

파이썬은 -v 옵션으로 시작됩니다

3.8.4. 사용자 정의

3.8.4.1. INI 파일을 통한 사용자 정의

런처는 두 개의 .ini 파일을 검색합니다 - 현재 사용자의 《application data》 디렉터리 (즉, CSIDL_LOCAL_APPDATA 를 사용하여 윈도우 함수 SHGetFolderPath 를 호출하여 반환

된 디렉터리)에 있는 py.ini와 런처와 같은 디렉터리에 있는 py.ini. 런처의 〈콘솔〉 버전(즉 py.exe)과 〈윈도우〉 버전(즉 pyw.exe) 모두에 같은 .ini 파일이 사용됩니다.

《응용 프로그램 디렉터리》에 지정된 사용자 정의가 실행 파일 옆에 있는 사용자 지정보다 우선하므로, 런처 옆에 있는 .ini 파일에 대한 쓰기 권한이 없는 사용자는 전역 .ini 파일에서 명령을 재정의할 수 있습니다.

3.8.4.2. 기본 파이썬 버전 사용자 정의

때에 따라, 버전 한정자를 명령에 포함하여 명령에서 사용할 파이썬 버전을 지정할 수 있습니다. 버전 한정자는 주 버전 번호로 시작하며 선택적으로 마침표(〈.〉)와 부 버전 지정자가 올 수 있습니다. 또한 《-32》나 《-64》를 추가하여 32비트나 64비트 구현을 요청할지를 지정할 수 있습니다.
예를 들어, #!python의 셔뱅 줄에는 버전 한정자가 없는 반면, **#!python3에는 주 버전 만 지정하는 버전 한정자가 있습니다.**

명령에 버전 한정자가 없으면, 환경 변수 PY_PYTHON을 설정하여 기본 버전 한정자를 지정할 수 있습니다. 설정되지 않으면, 기본값은 《3》입니다. 변수는 《3》, 《3.7》, 《3.7-32》 또는 《3.7-64》와 같이 명령 줄에서 전달할 수 있는 모든 값을 지정할 수 있습니다. (《-64》 옵션은 파이썬 3.7 이상에 포함된 런처에서만 사용할 수 있음에 유의하십시오.)

부 버전 한정자가 없으면, 환경 변수 PY_PYTHON{major}(여기서 {major}는 위에서 결정된 현재 주 버전 한정자입니다)를 설정하여 전체 버전을 지정할 수 있습니다. 그러한 옵션이 없으면, 런처는 설치된 파이썬 버전을 열거하고 주 버전에 대해 발견된 최신 부 릴리스를 사용합니다. 이것은 보장되지는 않지만, 해당 제품군에서 가장 최근에 설치된 버전일 가능성이 높습니다.
같은 (major.minor) 파이썬 버전의 32비트와 64비트 구현이 모두 설치된 64비트 윈도우에서는 항상 64비트 버전이 선호됩니다. 이는 32비트와 64비트 런처 구현 모두에 해당합니다 - 32비트 런처는 가능하면 지정된 버전의 64비트 파이썬 설치를 실행하는 것을 선호합니다. 따라서 런처의 동작은 PC에 설치된 버전만 알고 설치 순서와 관계없이 (즉, 32비트나 64비트 버전의 파이썬과 해당 런처가 마지막에 설치되었는지 알지 못하고) 예측할 수 있습니다). 위에서 언급했듯이, 선택적 《-32》나 《-64》 접미사를 버전 지정자에 사용하여 이 동작을 변경할 수 있습니다.
예:
관련 옵션이 설정되지 않으면, python과 python2 명령은 설치된 최신 파이썬 2.x 버전을 사용하고 python3 명령은 설치된 최신 파이썬 3.x를 사용합니다.
python3.1과 python2.7 명령은 버전이 완전히 지정되어 있어서 옵션을 전혀 참조하지 않습니다.
PY_PYTHON=3이면, python과 python3 명령은 모두 최신 설치된 파이썬 3 버전을 사용합니다.
PY_PYTHON=3.1-32이면, python 명령은 3.1의 32비트 구현을 사용하는 반면 python3 명령은 가장 최근에 설치된 파이썬을 사용합니다 (주 버전이 지정되었기 때문에 PY_PYTHON은

전혀 고려되지 않았습니다).
PY_PYTHON=3이고 PY_PYTHON3=3.1이면, python과 python3 명령은 모두 구체적으로 3.1을 사용합니다.
환경 변수 외에도, 런처에서 사용하는 .INI 파일에서 같은 설정을 구성할 수 있습니다. INI 파일의 섹션은 [defaults]라고 하며 키 이름은 선행 PY_ 접두어가 없는 환경 변수와 같습니다 (그리고 INI 파일의 키 이름은 대소 문자를 구분하지 않음에 유의하십시오.) 환경 변수의 내용은 INI 파일에 지정된 것을 재정의합니다.
예를 들면:

PY_PYTHON=3.1을 설정하는 것은 다음을 포함하는 INI 파일과 동등합니다:

```
[defaults]
python=3.1
```

PY_PYTHON=3과 PY_PYTHON3=3.1을 설정하는 것은 다음을 포함하는 INI 파일과 동등합니다:

```
[defaults]
python=3
python3=3.1
```

3.8.5. 진단

환경 변수 PYLAUNCH_DEBUG가 설정되면 (어떤 값이건), 런처는 진단 정보를 stderr(즉, 콘솔)에 인쇄합니다. 이 정보는 동시에 상세하면서도 간결하게 관리되지만, 어떤 버전의 파이썬을 찾았는지, 특정 버전이 선택된 이유 및 대상 파이썬을 실행하는 데 사용된 정확한 명령 줄을 볼 수 있습니다.

3.9. 모듈 찾기

파이썬은 일반적으로 라이브러리(및 그래서 site-packages 폴더)를 설치 디렉터리에 저장합니다. 따라서, 파이썬을 C:\Python\에 설치했다면 기본 라이브러리는 C:\Python\Lib\에 있고 제삼자 모듈은 C:\Python\Lib\site-packages\에 저장되어야 합니다.
sys.path를 완전히 재정의하려면, DLL(python37._pth)이나 실행 파일(python._pth)과 같은 이름으로 ._pth 파일을 만들고 sys.path에 추가할 경로마다 한 줄씩 지정합니다. DLL 이름을 기반으로 하는 파일은 실행 파일을 기반으로 하는 파일을 재정의하므로, 원한다면 런타임을 로드하는 모든 프로그램에 대해 경로를 제한 할 수 있습니다.
파일이 존재할 때, 모든 레지스트리와 환경 변수가 무시되고, 격리 모드(isolated mode)가 활성화되고, 파일의 한 줄이 import site를 지정하지 않는 한 site를 임포트 하지 않습니다. 빈 경로와 #으로 시작하는 줄은 무시됩니다. 각 경로는 파일 위치에 대해 절대적이거나 상대적일 수 있습니다. site 이외의 임포트 문은 허용되지 않으며, 임의 코드를 지정할 수 없습니다.
import site가 지정될 때, .pth 파일(선행 밑줄 없는)은 site 모듈에서 정상적으로 처리됨에 유의하십시오.

._pth 파일이 없을 때, 윈도우에서 sys.path를 채우는 방법은 다음과 같습니다:
시작에 현재 디렉터리에 해당하는 빈 항목이 추가됩니다.
환경 변수에 설명된 대로, 환경 변수 PYTHONPATH가 존재하면, 해당 항목이 다음에 추가됩니다. 윈도우에서, 이 변수의 경로는 드라이브 식별자(C:\ 등)에 사용되는 콜론과 구별하기 위해 세미콜론으로 구분되어야 합니다.
추가 《응용 프로그램 경로》는 HKEY_CURRENT_USER와 HKEY_LOCAL_MACHINE 하이브 모두의 아래에 \SOFTWARE\Python\PythonCore{version}\PythonPath의 하위 키로 레지스트리에 추가될 수 있습니다. 세미콜론으로 구분된 경로 문자열을 기본값으로 사용하는 하위 키는 각 경로가 sys.path에 추가되도록 합니다. (알려진 모든 설치 프로그램은 HKLM 만 사용하므로, HKCU는 일반적으로 비어 있음에 유의하십시오.)
환경 변수 PYTHONHOME이 설정되면, 《파이썬 홈》으로 간주합니다. 그렇지 않으면, 메인 파이썬 실행 파일의 경로를 사용하여 《랜드마크 파일》(Lib\os.py나 pythonXY.zip)을 찾아 《파이썬 홈》을 추론합니다. 파이썬 홈이 발견되면, sys.path에 추가되는 관련 하위 디렉터리(Lib, plat-win 등)는 해당 폴더를 기반으로 합니다. 그렇지 않으면, 핵심 파이썬 경로가 레지스트리에 저장된 PythonPath에서 구성됩니다.
파이썬 홈을 찾을 수 없고, 환경에 PYTHONPATH가 지정되어 있지 않고, 레지스트리 항목을 찾을 수 없으면, 상대 항목의 기본 경로(예를 들어 .\Lib;.\plat-win 등)가 사용됩니다.
pyvenv.cfg 파일이 메인 실행 파일과 함께 또는 실행 파일보다 한 수준 위의 디렉터리에서 발견되면, 다음 변형이 적용됩니다:
home이 절대 경로이고 PYTHONHOME이 설정되지 않으면, 홈 위치를 추론할 때 메인 실행 파일에 대한 경로 대신 이 경로가 사용됩니다.
이 모든 것의 최종 결과는 다음과 같습니다:
python.exe 또는 기본 파이썬 디렉터리 (설치된 버전 또는 PCbuild 디렉터리에서 직접)에서 다른 .exe를 실행할 때 핵심 경로가 추론되고 레지스트리의 핵심 경로가 무시됩니다. 레지스트리의 다른 《응용 프로그램 경로》는 항상 읽습니다.
파이썬이 다른 .exe(다른 디렉터리, COM을 통한 내장, 등)에서 호스팅 될 때, 《파이썬 홈》이 추론되지 않아서, 레지스트리의 핵심 경로가 사용됩니다. 레지스트리의 다른 《응용 프로그램 경로》는 항상 읽힙니다.
파이썬이 홈을 찾을 수 없고 레지스트리 값이 없으면 (고정된(frozen) .exe, 아주 이상한 설치 설정), 일부 기본 (하지만 상대) 경로를 얻게 됩니다.
파이썬을 응용 프로그램이나 배포에 번들로 포함하려는 사용자를 위해, 다음 조언은 다른 설치와의 충돌을 방지합니다:
포함할 디렉터리가 포함된 실행 파일과 함께 ._pth 파일을 포함합니다. 이것은 레지스트리와 환경 변수에 나열된 경로를 무시하고, import site가 나열되지 않는 한 site도 무시합니다.
여러분 자신의 실행 파일에서 python3.dll이나 python37.dll을 로드하면, Py_Initialize() 전에 Py_SetPath()나 (적어도) Py_SetProgramName() 을 명시적으로 호출하십시오.
응용 프로그램에서 python.exe를 시작하기 전에 PYTHONPATH를 지우거나 덮어쓰고 PYTHONHOME를 설정하십시오.
이전 제안을 사용할 수 없으면 (예를 들어, 사용자가 python.exe를 직접 실행할 수 있는 배포판이면), 랜드마크 파일(Lib\os.py)이 설치 디렉터리에 있도록 하십시오. (ZIP 파일 내에서는 감지되지 않지만, 대신 올바른 이름의 ZIP 파일은 감지됨에 유의하십시오.)

이렇게 하면 시스템 전체 설치의 파일이 응용 프로그램과 함께 번들로 제공되는 표준 라이브러리의 복사본보다 우선하지 않습니다. 그렇지 않으면, 사용자가 여러분의 응용 프로그램을 사용하는 데 문제가 발생할 수 있습니다. 다른 제안은 레지스트리의 비표준 경로와 사용자 site-packages에 여전히 취약할 수 있어서, 첫 번째 제안이 가장 좋음에 유의하십시오.

버전 3.6에서 변경:

._pth 파일 지원을 추가하고 pyvenv.cfg에서 applocal 옵션을 제거합니다.

실행 파일에 직접 인접할 때 잠재적인 랜드마크로 pythonXX.zip을 추가합니다.

버전 3.6부터 폐지:

Modules(PythonPath가 아닙니다) 아래의 레지스트리에 지정된 모듈은 importlib.machinery.WindowsRegistryFinder 로 임포트 할 수 있습니다. 이 파인더는 윈도우 3.6.0과 이전 버전에서 활성화되지만, 향후에는 sys.meta_path에 명시적으로 추가해야 할 수도 있습니다.

3.10. 추가 모듈

파이썬이 모든 플랫폼 간에 이식성 있는 것을 목표로 하지만, 윈도우에만 고유한 기능이 있습니다. 이러한 기능을 사용하기 위한 표준 라이브러리와 외부에 있는 두 개의 모듈과 스니펫(snippets)이 존재합니다.

윈도우 특정 표준 모듈은 MS 윈도우 특정 서비스에 설명되어 있습니다.

3.10.1. PyWin32

Mark Hammond의 PyWin32 모듈은 고급 윈도우 특정 지원을 위한 모듈 모음입니다. 여기에는 다음을 위한 유틸리티가 포함됩니다:

Component Object Model (COM)

Win32 API 호출

레지스트리

이벤트 로그

Microsoft Foundation Classes (MFC) 사용자 인터페이스

PythonWin은 PyWin32와 함께 제공되는 샘플 MFC 응용 프로그램입니다. 디버거가 내장된 내장할 수 있는 IDE입니다.

더 보기

Win32 How Do I…?

저자: Tim Golden

Python and COM

저자: David와 Paul Boddie

3.10.2. cx_Freeze

cx_Freeze is a distutils extension (see Distutils 확장하기) which wraps Python scripts into executable Windows programs (*.exe files). When you have done this, you can

distribute your application without requiring your users to install Python.

3.10.3. WConio

파이썬의 고급 터미널 처리 계층인 curses가 유닉스 계열 시스템으로 제한되기 때문에, 윈도우 전용 라이브러리도 있습니다: Windows Console I/O for Python.
WConio는 텍스트 사용자 인터페이스를 만드는 데 사용되는 Turbo-C의 CONIO.H용 래퍼입니다.

3.11. 윈도우에서 파이썬 컴파일하기

CPython을 직접 컴파일하려면, 먼저 소스를 가져와야 합니다. 최신 릴리스의 소스를 다운로드하거나 최신 버전을 체크아웃할 수 있습니다.
소스 트리에는 공식 파이썬 릴리스를 빌드하는 데 사용되는 컴파일러인 Microsoft Visual Studio 2015 용 빌드 솔루션과 프로젝트 파일이 포함되어 있습니다. 이 파일들은 PCbuild 디렉터리에 있습니다.
빌드 프로세스에 대한 일반 정보는 PCbuild/readme.txt를 확인하십시오.
확장 모듈에 대해서는, 윈도우에서 C와 C++ 확장 빌드하기를 참조하십시오.
더 보기
Python + Windows + distutils + SWIG + gcc MinGW
or 《Creating Python extensions in C/C++ with SWIG and compiling them with MinGW gcc under Windows》 or 《Installing Python extension with distutils and without Microsoft Visual C++》 by Sébastien Sauvage, 2003

3.12. 기타 플랫폼

파이썬의 지속적인 개발로 인해, 이전에 지원되던 일부 플랫폼은 더는 지원되지 않습니다 (사용자나 개발자 부족으로 인해). 지원되지 않는 모든 플랫폼에 대한 자세한 내용은 PEP 11을 확인하십시오.
윈도우 CE는 계속 지원됩니다.
Cygwin 설치 프로그램은 파이썬 인터프리터도 설치하도록 제안합니다 (참조: Cygwin package source, Maintainer releases).
사전 컴파일된 설치 프로그램이 있는 플랫폼에 대한 자세한 정보는 Python for Windows를 참조하십시오.